**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH**
**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN**
**KHOA KHOA HỌC MÁY TÍNH**

# MÔN HỌC: Cấu Trúc Dữ Liệu Và Giải Thuật Nâng Cao

# MÃ MÔN HỌC: CS523.Q21

# Ứng Dụng Từ Điển

Giảng viên hướng dẫn:
Thạc Sĩ Nguyễn Thanh Sơn

Sinh viên thực hiện:
Ngô Hồng Vinh – 24522016

ꕥ Tp. Hồ Chí Minh, 04/2026 ꕥ

**NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN**

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

*……, ngày…….....tháng……năm 2026*

**Người nhận xét**

*(Ký tên và ghi rõ họ tên)*

# I. Tổng Quan

## 1. Giới Thiệu

Đây là ứng dụng đánh tra từ điển. RadixTree sẽ đóng vai trò như chỉ mục tìm kiếm phục vụ các thao tác tìm, xóa, sửa. Với Trie thông thường, mỗi ký tự được lưu trên một cạnh riêng biệt nên chỉ cần khởi tạo con trỏ đến 26 kí tự là xong. Tuy nhiên, với Radix Tree, do mỗi node lưu trữ một chuỗi con (substring), nên khi xuất hiện sự trùng lặp một phần giữa các chuỗi, cây cần thực hiện thao tác **tách node (split)** để duy trì cấu trúc đúng.

Cụ thể, khi chèn một chuỗi mới, nếu chuỗi này có chung một tiền tố với chuỗi đã tồn tại nhưng không trùng hoàn toàn, node hiện tại sẽ được tách thành nhiều phần: phần tiền tố chung được giữ lại làm node cha, còn các phần hậu tố khác nhau sẽ trở thành các node con riêng biệt. Nhờ đó, cây vẫn đảm bảo rằng mỗi nhánh chỉ đại diện cho một đoạn chuỗi không trùng lặp.

Ngược lại, khi xóa một chuỗi, nếu sau khi xóa mà một node chỉ còn duy nhất một node con, hai node này sẽ được **gộp lại (merge)** bằng cách nối các chuỗi con của chúng. Điều này giúp duy trì tính chất nén của Radix Tree, tránh việc tồn tại các node trung gian không cần thiết.

Trước hết ta định nghĩa các Node:

```python
class Node:
    def __init__(self, label=""):
        self.label = label        # substring
        self.children = {}        # {first_char: Node}
        self.is_end = False
```

Sau đó, ta định nghĩa RadixTree, cùng với phương thức tìm tiền tố chung dài nhất. Ý tưởng là duyệt từ kí tự 0 đến kí tự ngắn nhất của 1 trong 2 xâu, nếu cả hai xâu có kí tự giống nhau thì tiếp tục duyệt:

```python
class RadixTree:
    def __init__(self):
        self.root = Node()

    # ===== Longest Common Prefix =====
    def _lcp(self, s1, s2):
        i = 0
        while i < min(len(s1), len(s2)) and s1[i] == s2[i]:
            i += 1
        return s1[:i]
```

Kế đến, ta định nghĩa phương thức tìm kiếm. Nếu tồn tại một node con mà chuỗi đầu vào bắt đầu bằng label của nó, ta coi như đã tìm được một phần khớp. Khi đó, phần

chuỗi đã khớp sẽ được dịch sang phải một đoạn tương ứng với độ dài của label, và hàm đệ quy tiếp tục được gọi trên node con đó với phần chuỗi còn lại:

```python
# ===== SEARCH =====
def search(self, word):
    return self._search(self.root, word)

def _search(self, node, word):
    for child in node.children.values():
        if word.startswith(child.label):
            if word == child.label:
                return child.is_end
            return self._search(child, word[len(child.label):])
    return False
```

Ta tiếp tục định nghĩa phương thức chèn. Tương tự như quá trình tìm kiếm, thao tác chèn trong Radix Tree cũng bắt đầu bằng việc duyệt cây theo cơ chế đệ quy và so khớp tiền tố giữa chuỗi cần chèn và các node hiện có. Tại mỗi bước, nếu tìm được node con có label là tiền tố của chuỗi, ta tiếp tục dịch phải phần đã khớp và đệ quy xuống node đó. Trong quá trình này, nếu xảy ra trường hợp chuỗi cần chèn và label của node chỉ trùng nhau một phần (tức là có tiền tố chung nhưng không hoàn toàn), ta sẽ thực hiện thao tác **tách node (split)**. Cụ thể, phần tiền tố chung được tách ra thành một node mới, còn hai phần hậu tố khác nhau sẽ trở thành các node con riêng biệt. Điều này đảm bảo cấu trúc nén của Radix Tree vẫn được duy trì. Ngược lại, nếu không xảy ra xung đột hay trùng lặp một phần nào, tức là không tồn tại node con phù hợp, ta chỉ cần chèn trực tiếp một node mới chứa toàn bộ phần chuỗi còn lại vào cây.

```python
def insert(self, word):
    self._insert(self.root, word)

def _insert(self, node, word):
    for child_key in list(node.children.keys()):
        child = node.children[child_key]
        prefix = self._lcp(child.label, word)

        if not prefix:
            continue

        # Case 1: trùng hoàn toàn node
        if prefix == child.label and prefix == word:
            child.is_end = True
            return

        # Case 2: node là prefix → đi tiếp
        if prefix == child.label:
            self._insert(child, word[len(prefix):])
            return

        # ===== CASE QUAN TRỌNG: SPLIT =====
        new_node = Node(prefix)

        # phần còn lại của node cũ
        child.label = child.label[len(prefix):]

        # gắn node cũ làm con
        new_node.children[child.label[0]] = child

        # phần còn lại của word mới
        new_child = Node(word[len(prefix):])
        new_child.is_end = True
        new_node.children[new_child.label[0]] = new_child

        # thay thế node cũ bằng node mới
        node.children[prefix[0]] = new_node
        del node.children[child_key]
        return

    # không có prefix → thêm mới
    new_node = Node(word)
    new_node.is_end = True
    node.children[word[0]] = new_node
```

Ta cuối cùng định nghĩa quá trình xóa. Tương tự như tìm kiếm và chèn, thao tác xóa trong Radix Tree cũng bắt đầu bằng việc duyệt cây theo cơ chế đệ quy và so khớp tiền tố giữa chuỗi cần xóa và các node hiện có. Tại mỗi bước, nếu tìm được node con có label là tiền tố của chuỗi, ta tiếp tục "tiêu thụ" phần đã khớp và đệ quy xuống node tương ứng. Khi chuỗi được khớp hoàn toàn, node tương ứng sẽ được cập nhật bằng cách bỏ đánh dấu kết thúc từ. Sau đó, cây sẽ được kiểm tra lại để đảm bảo tính chất nén của Radix Tree:

- Nếu node không còn là điểm kết thúc và không có node con nào, nó sẽ bị xóa khỏi cây.
- Nếu node không còn là điểm kết thúc và chỉ có một node con, hai node này sẽ được **gộp lại (merge)** bằng cách nối các chuỗi con của chúng.

Quá trình này giúp loại bỏ các node trung gian không cần thiết và duy trì cấu trúc nén đặc trưng của Radix Tree.

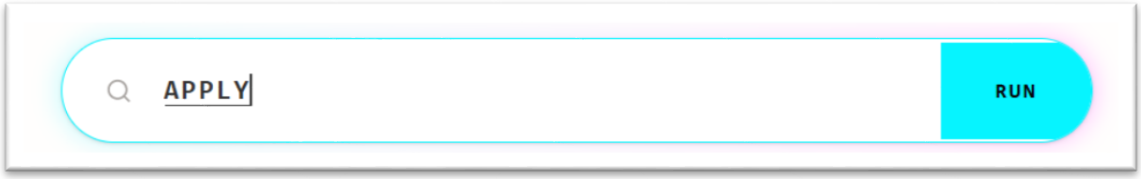
```
def delete(self, word):
    self._delete(self.root, word)

def _delete(self, node, word):
    for key in list(node.children.keys()):
        child = node.children[key]

        if word.startswith(child.label):

            # tìm đúng node
            if word == child.label:
                child.is_end = False

            else:
                self._delete(child, word[len(child.label):])

            # ===== MERGE =====
            # nếu child không phải end và chỉ có 1 con
            if not child.is_end and len(child.children) == 1:
                grandchild = next(iter(child.children.values()))
                child.label += grandchild.label
                child.is_end = grandchild.is_end
                child.children = grandchild.children

            # nếu child rỗng → xóa
            if not child.is_end and len(child.children) == 0:
                del node.children[key]

            return
```

## II. Hướng Dẫn Sử Dụng

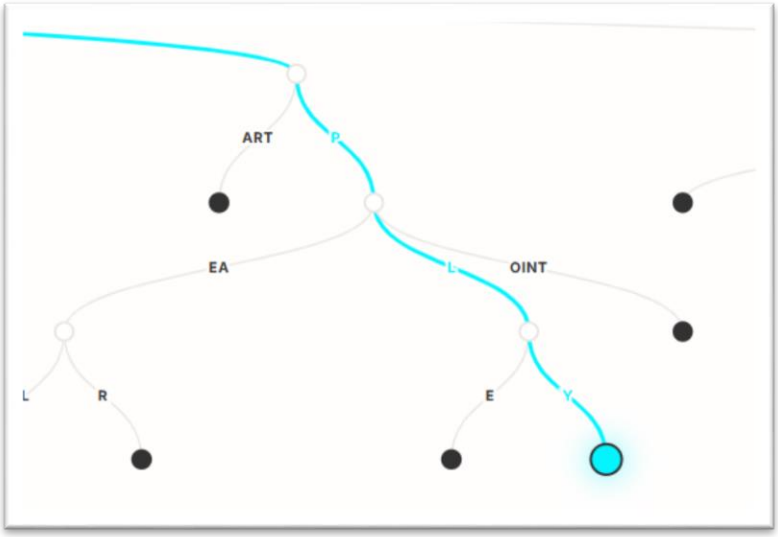
Điều hướng đến đường dẫn sau: https://cs-523-radix-tree.vercel.app/. Sau khi, truy cập đường dẫn, ta sẽ thấy trang web sau:

1. **Tìm Kiếm**

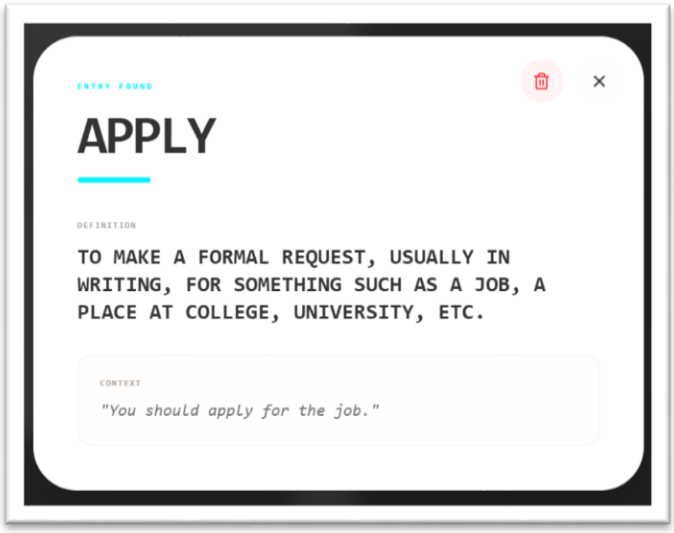
Để thực hiện tìm kiếm, ta nhập từ ta muốn tìm kiếm (trong ví dụ này là "APPLY"), sau đó ta bấm RUN:




Trong quá trình chạy, trình mô phỏng duyệt Radix Tree bằng cách lần lượt so khớp **tiền tố chung dài nhất (longest common prefix)** giữa chuỗi đầu vào và nội dung được lưu tại các node. Ở mỗi bước, node hiện tại sẽ được làm nổi bật để biểu diễn trạng thái duyệt. Quá trình này tiếp tục cho đến khi chuỗi được khớp hoàn toàn hoặc không còn node phù hợp, từ đó xác định kết quả của thao tác.

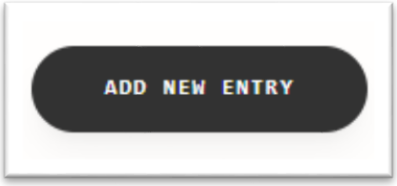



Sau khi tìm kiếm thành công, từ tương ứng sẽ hiện lên với định nghĩa trong tiếng anh:

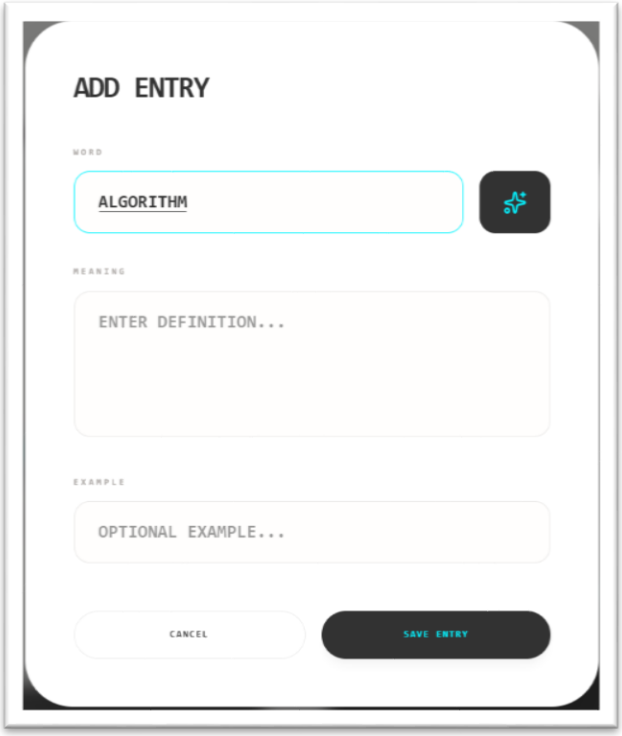



## 2. Chèn

Để thêm 1 từ, nhấn ADD NEW ENTRY ở góc trên bên phải:

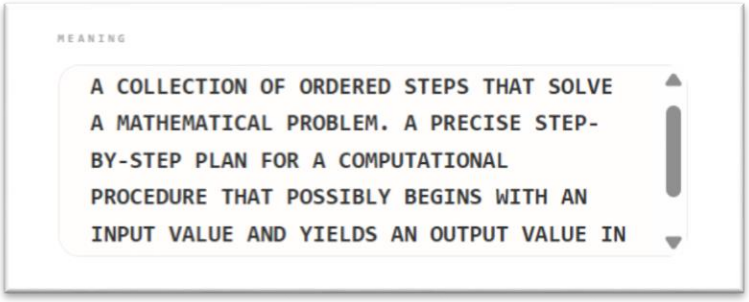



Sau khi nhấn góc trên bên phải, ta điền từ ta muốn thêm, ở đây ta điền từ Algorithm:



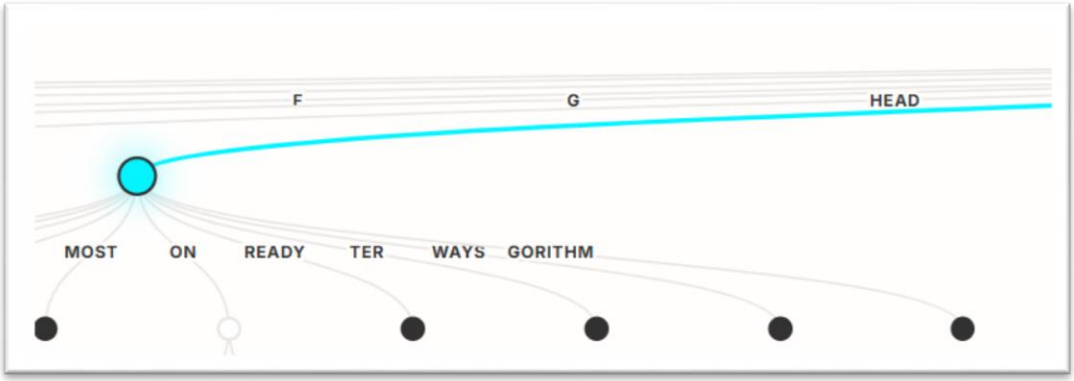

Nếu ta không muốn tự ghi định nghĩa, có thể dùng API Dictionary bằng cách bấm nút này:

Lúc này, nó sẽ tự động thêm nghĩa vô:

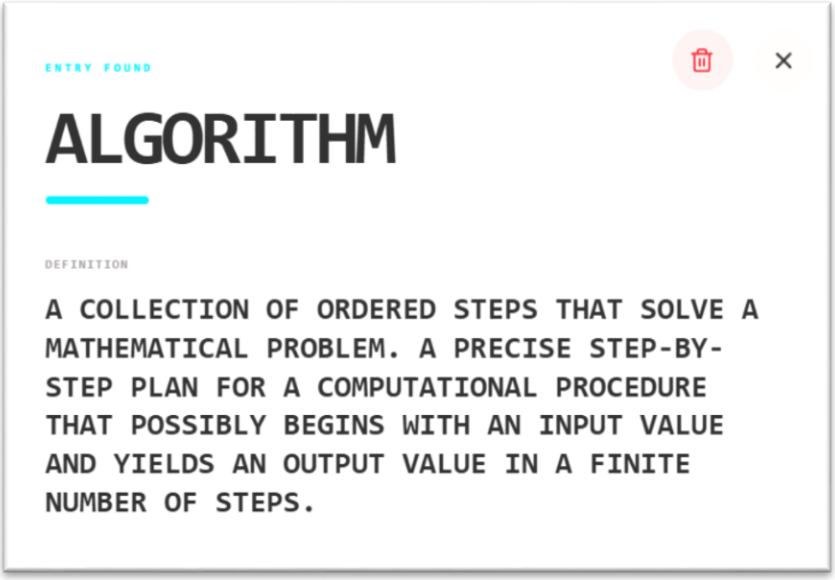



Và nhấn nút SAVE ENTRY:

Lúc đó, quy trình sẽ hiển thị các bước và mô phỏng trên trình mô phỏng:



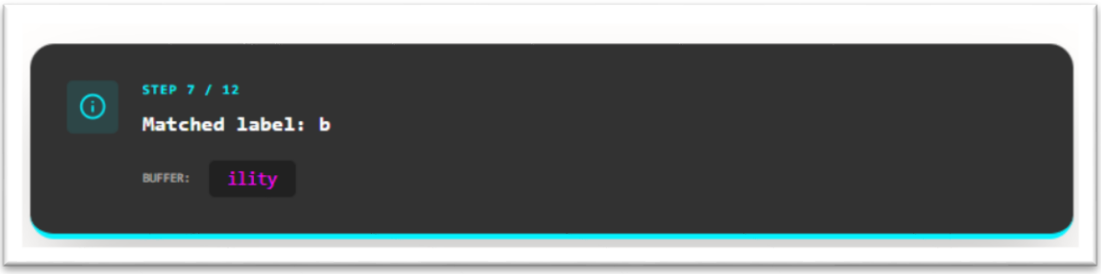
Cuối cùng, sau khi hoàn tất sẽ hiển thị lại từ vừa điền:



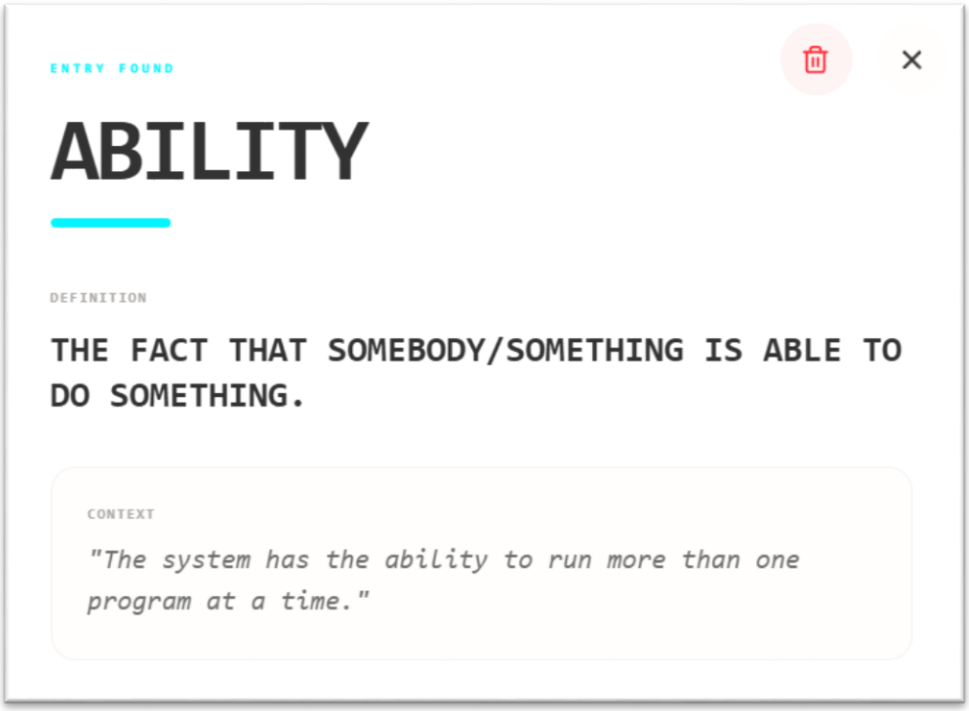

### 3. Xóa

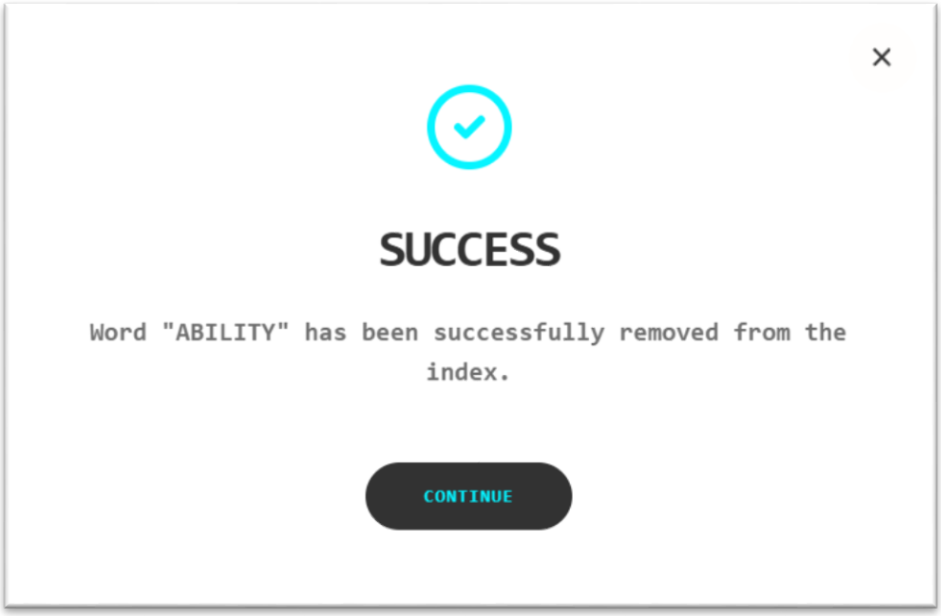
Để thực hiện xóa, ta kéo xuống dưới. Ở phần STORED WORDS, ta lựa chọn từ ta muốn xóa, ở đây là ABILITY:

Lúc này, nó sẽ bắt đầu quá trình tìm kiếm và xóa:




Sau khi tìm xong nó sẽ hiển thị lại, lúc này ta nhấn xóa 1 lần nữa (Vì có thể ta bấm nhầm nên cần 2 lần).




Sau khi bấm xong, chúc mừng ta đã xóa thành công: